3. 그래프 (2) CSED331 · 알고리즘 전체 홈↗

이 페이지

1. 최단 경로 문제와 DFS의 한계
2. Dijkstra 알고리즘
3. 음수 간선과 Bellman-Ford
4. 보충 · Shortest Reliable Paths (간선 수 제한 최단 경로)
5. DAG — 위상 순서와 최단 경로
6. 동적 계획법으로 풀리는 최단 경로 문제
7. 연습문제

그래프 (2) — 최단 경로

2026-09-21 (월) 뒷부분 · 09-23 (수) · 슬라이드 Graph Algorithms (p.19–36, 녹취 없음, 교수 marked PDF)

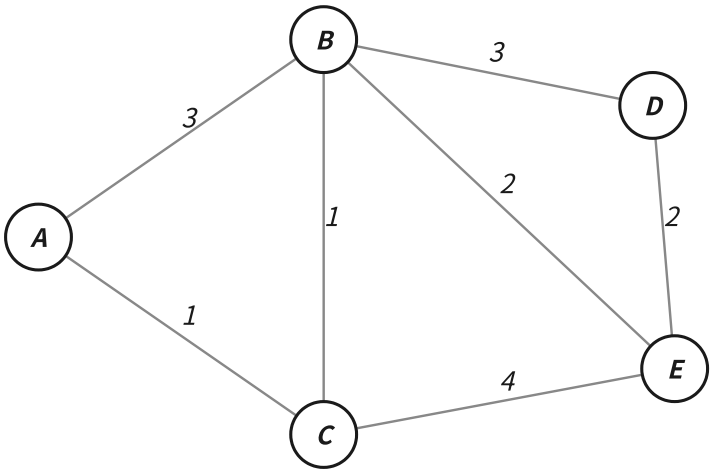
최단 경로 문제와 DFS의 한계

- **distance** — 두 노드 사이 shortest path의 길이 (경로 위 간선 길이의 합). 간선 길이는 3, 1, −2처럼 제각각
- **DFS 트리는 거리를 반영하지 못함** — 슬라이드 예: S와 C는 간선 하나인데 DFS 트리에선 S → A → B → C (3칸). C: 실제 1 vs 트리 3, E: 실제 1 vs 트리 2
- 간선 길이가 전부 1이면 BFS가 최단 경로 (앞 장). 실로 노드를 연결한 물리 모델은 거리를 그대로 반영

가중치가 있는 그래프의 최단 경로를 위한 두 가지 접근

- **1차: dummy node** — 길이 ℓ 간선을 $\ell - 1$개의 dummy 노드로 바꾸고 BFS. 답은 맞지만 길이가 10^6이면 dummy 세느라 시간을 다 씀. 정수가 아니거나 음수면 불가. 모든 간선 길이가 양의 정수 $\ell \geq 1$이면, 길이 ℓ인 간선을 $\ell-1$개의 dummy 정점을 거치는 ℓ개 단위 간선으로 대체할 수 있다. 가중치가 0이거나 음수이거나 정수가 아닌 경우에는 이 변환을 그대로 적용할 수 없다
- **2차: alarm clock** — alarm(s) = 0, 나머지 ∞. 시각 T에 노드 u에서 알람이 울리면 $\mathrm{dist}(s,u) = T$ 확정, 이웃 v마다 alarm(v) = min{alarm(v), $T + \ell(u,v)$}. **가장 먼저 울리는 알람 찾기 = priority queue** 이 방식은 가중치가 정수일 필요는 없지만, 모든 간선 가중치가 음수가 아니어야 한다.

alarm clock 아이디어 — 길이 ℓ 간선마다 시각 T + ℓ 에 알람



알람이 울린 순서

A — 시각 0

C — 시각 1

B — 시각 2

E — 시각 4

D — 시각 5

B는 A 경유 3 → C 경유 2로 내려감

가장 먼저 울리는 알람 찾기 = priority queue → Dijkstra

Dijkstra 알고리즘

1. 조건

- 입력: 방향 또는 무방향 가중치 그래프와 출발점 s.
- 표준 Dijkstra 알고리즘의 조건은 모든 간선 가중치가 음수가 아닌 것, 즉 $\ell(u, v) \geq 0$이다. 양수 가중치만 가정해도 알고리즘은 정확하지만, 가중치가 0인 간선도 허용된다.
- 음수 간선이 있으면 최솟값 정점의 거리를 확정하는 논리가 깨질 수 있다. 나중에 더 짧은 경로가 발견될 수 있기 때문이다.
- 출력: 출발점 s에서 도달 가능한 모든 정점의 최단 거리 $dist$.

---

2. 절차

알람 시계(alarm clock) 아이디어를 priority queue로 구현한다. 출발점의 거리는 0, 나머지 정점의 거리는 ∞로 초기화한다.

Priority queue 연산

연산	의미
insert(H, v)	원소 추가

decreasekey(H, v)	key가 줄었을 때 위치 조정
deletemin(H)	key가 가장 작은 원소를 꺼냄
makequeue(V)	정점 집합 V로 큐 생성, key는 dist

반복 절차

1. $u = \text{deletemin}(H)$로 거리가 가장 작은 정점을 꺼내 확정한다.
2. u에서 나가는 각 간선 (u, v)에 대해 검사한다.
3. $dist(v) > dist(u) + \ell(u, v)$이면 다음과 같이 갱신한다.

$$dist(v) = dist(u) + \ell(u, v)$$

$$prev(v) = u$$

이후 $\text{decreasekey}(H, v)$를 수행한다.

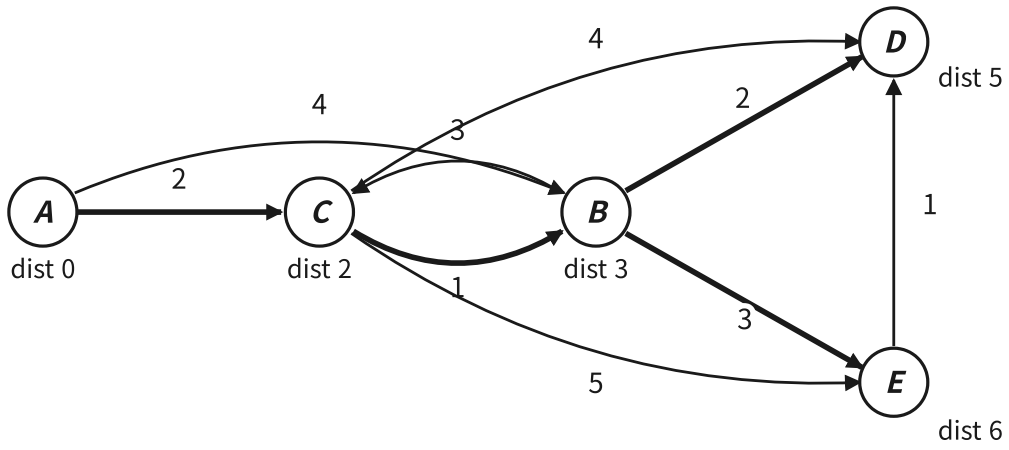
꺼낸 정점의 dist가 ∞이면, 그 정점과 아직 확정되지 않은 나머지 정점에는 도달할 수 없으므로 알고리즘을 종료할 수 있다.

prev와 shortest path tree

- $prev(v)$: $dist(v)$를 마지막으로 낮춘 정점.
- $prev$를 거꾸로 따라가면 해당 정점까지의 최단 경로를 얻는다.
- 모든 $prev$ 간선을 모으면 shortest path tree가 된다.

손계산: 슬라이드 22 그래프

Dijkstra (시작 A) — 굵은 선 = shortest path tree (A→C, C→B, B→D, B→E)



확정 순서 A, C, B, D, E (dist 0, 2, 3, 5, 6으로 증가)
A → E 최단 = A→C→B→E = 6

단계	deletemin (확정)	decreasekey	A	B	C	D	E
0	–	–	$dist(A) = 0$	∞	∞	∞	∞
1	A (0)	B = 4, C = 2	0	4	2	∞	∞
2	C (2)	B = 3, D = 6, E = 7	0	3	2	6	7
3	B (3)	D = 5, E = 6	0	3	2	5	6
4	D (5)	나가는 간선 없음	0	3	2	5	6
5	E (6)	갱신 없음 (E→D: 6+1=7 > 5)	0	3	2	5	6

- 확정 순서: A, C, B, D, E.
- 확정된 거리: 0, 2, 3, 5, 6. 꺼낸 정점의 $dist$는 이후 바뀌지 않는다.
- B는 A 경유 4에서 C 경유 3으로 내려간 뒤 확정된다. 따라서 $prev(B)$는 A에서 C로 바뀐다.
- 최종 $prev$: A는 nil, B는 C, C는 A, D는 B, E는 B.
- A에서 E까지의 최단 경로는 $A \to C \to B \to E$이며 길이는 6이다.
- 연산 횟수: $\mathrm{deletemin}$ 5번, $\mathrm{decreasekey}$ 7번, 간선 검사 9번.
- 출발점을 B로 바꾸면 확정 순서는 B, D, C, E이고 거리는 B 0, D 2, C 3, E 3, A ∞이다. A로 들어오는 간선이 없으므로 A에는 도달할 수 없다.

3. 정확성

핵심 관찰은 최단 경로를 이미 알려진 최단 경로에 간선 하나를 추가하는 방식으로 확장할 수 있다는 것이다. 현재까지 거리가 확정된 정점 집합을 S라고 하자. S에서 바깥으로 나가는 간선으로 확장할 수 있는 후보 중 가장 짧은 경로의 끝점이 다음 확정 정점이 된다.

Invariant: 간선 길이가 음수가 아닐 때, 최단 경로의 각 prefix 정점 x는 d(s, x) ≤ d(s, v)를 만족한다. 간선 가중치가 모두 양수라면 v 이전 정점에 대해 d(s, x) < d(s, v)이다. 0 가중치 간선이 있으면 앞선 정점의 거리가 v와 같을 수 있지만 확정 논리는 유효하다.

왜 더 짧은 다른 경로가 없는가? 아직 확정되지 않은 정점 v로 가는 다른 경로를 생각하자. 이 경로는 어느 지점에서 처음으로 S를 벗어나며, 그때 간선 (x, y)를 지난다. 여기서 $x \in S, y \notin S$이다. x의 거리는 이미 최단 거리로 확정되었다. 따라서 이 경로의 길이는 x까지의 최단 거리와 간선 (x, y)의 가중치를 더한 값 이상이다. 이 값은 y의 후보 거리 이상이고, v는 후보 거리가 가장 작은 정점이므로 v의 후보 거리 이상이다. y 이후의 간선 가중치도 음수가 아니므로 이후 구간을 더해도 이 하한보다 짧아질 수 없다. 따라서 v보다 짧은 경로는 존재하지 않으며, $\mathrm{deletemin}$으로 꺼낸 정점의 거리를 확정해도 안전하다.

---

4. 복잡도

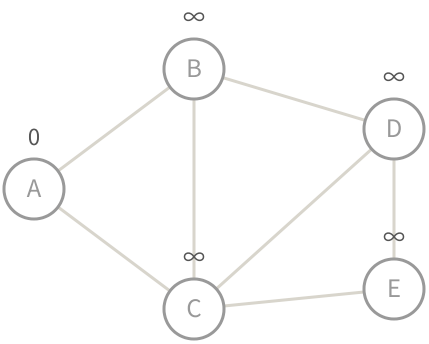
초기화에는 정점마다 $insert$를 수행하므로 $O(|V|)$회가 필요하다. 정점 확정은 $\mathrm{deletemin}$을 통해 정점마다 한 번 수행하며, 간선 검사는 전체 $O(|E|)$회이다. 성공적인 완화에 따른 $\mathrm{decreasekey}$는 최대 $O(|E|)$회이다. 슬라이드 22 예시에서는 $\mathrm{decreasekey}$ 7회, 간선 검사 9회이다.

자료구조	deletemin	insert·decreasekey	전체 시간
Array	$O(\|V\|)$	$O(1)$	$O(\|V\|^2 + \|E\|)$
Binary heap	$O(\log \|V\|)$	$O(\log \|V\|)$	$O((\|V\| + \|E\|) \log \|V\|)$
d-ary heap	$O(d \log_d \|V\|)$	$O(\log_d \|V\|)$	$O((\|E\| + \|V\|d) \log_d \|V\|)$
Fibonacci heap	Amortized $O(\log \|V\|)$	Amortized $O(1)$	$O(\|V\| \log \|V\| + \|E\|)$

- Binary heap은 깊이가 $O(\log |V|)$이므로 $insert, \mathrm{deletemin}, \mathrm{decreasekey}$의 연산 비용이 로그 시간이다.
- d-ary heap은 d를 $|E|/|V|$ 수준으로 잡으면 sparse·dense 그래프 모두에서 효율적일 수 있다.

- Fibonacci heap은 이론적으로 우수한 시간 복잡도를 제공하지만 구현이 복잡하고 상수 비용이 커서, 일반적인 구현에서는 이진 힙이나 d-ary heap이 더 자주 선택된다. 다만 Fibonacci heap의 이론적 분석과 특수한 활용 가치는 여전히 있다.
- Sparse 그래프에서는 $|E| = O(|V|)$이므로 array의 $O(|V|^2)$보다 heap의 $O(|V| \log |V|)$이 유리하다.
- Dense 그래프에서는 $|E| = O(|V|^2)$이므로 array의 $O(|V|^2)$가 binary heap의 $O(|V|^2 \log |V|)$보다 유리할 수 있다.
- 따라서 간선 수가 정점 수 수준이면 heap, 간선 수가 $|V|^2$ 수준이면 array가 유리하다.
- 모든 간선 가중치가 1인 그래프에서는 BFS가 최단 경로를 구할 수 있으며, 이 경우 우선순위 큐를 사용하는 Dijkstra보다 일반적으로 효율적이다. 반면 가중치가 서로 다른 그래프에서는 BFS가 일반적으로 최단 경로를 보장하지 않는다.

다익스트라 최단 경로 (Dijkstra)

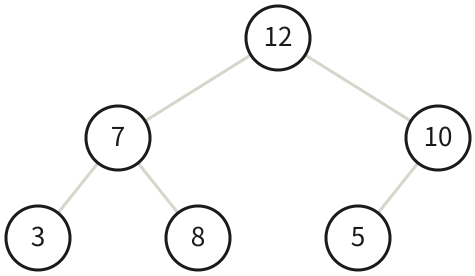


A에서 시작. dist[A]=0, 나머지 ∞. 매 단계 **미확정 정점 중 dist 최소**를 확정한다.
A=0 · B=∞ · C=∞ · D=∞ · E=∞

◀ 이전 1 / 14 다음 ▶

일반 힙 복습 — 최대 힙 예시(Dijkstra의 우선순위 큐와는 별개). 최대 힙에서는 부모의 key가 자식의 key 이상이며, extract-max는 가장 큰 key를 꺼낸다. 반면 Dijkstra에서는 가장 작은 dist를 가진 정점을 꺼내야 하므로 최소 우선순위 큐의 deletemin 연산을 사용한다.

힙 연산 (insert · extract-max)



배열: [12, 7, 10, 3, 8, 5]

최대 힙: 부모 ≥ 자식. 배열 [12, 7, 10, 3, 8, 5]는 이미 힙 조건을 만족한다.

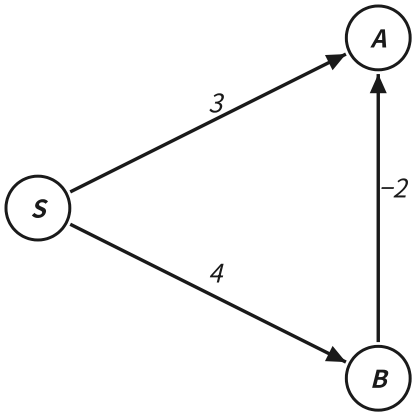
◀ 이전 1 / 8 다음 ▶

음수 간선과 Bellman-Ford

1. 조건

- Bellman-Ford는 음수 간선을 허용한다.
- 출발점에서 도달 가능한 음수 사이클이 없으면 도달 가능한 각 정점의 최단 거리를 계산할 수 있다.
- 출발점에서 도달 가능한 음수 사이클이 존재하면, 그 음수 사이클에서 도달할 수 있는 모든 정점의 최단 거리는 유한한 값으로 정의되지 않는다. 음수 사이클에서 도달할 수 없는 정점의 최단 거리는 이 이유만으로 정의되지 않는다고 할 수 없다.
- Dijkstra는 PQ에서 최소 정점을 꺼내 확정하지만 음수 간선이 있으면 확정 논리가 깨질 수 있다. Bellman-Ford는 최소 정점을 확정하지 않고 간선 (u, v)를 검사해 dist(u) + ℓ(u, v) < dist(v)이면 갱신하고 prev(v) = u로 갱신한다. 이 과정에 PQ는 필요하지 않다.
- 출발점에서도 도달 가능한 음수 사이클이 없다고 가정하면, 모든 유한한 dist(v)는 실제 경로의 길이이므로 최단 거리 d(s, v) 이상이다. 간선 완화는 dist(v)를 감소시키기만 하며, 이 조건에서는 최단 거리보다 작은 값을 만들지 않는다. 출발점에서도 도달 가능한 음수 사이클이 존재하면 해당 사이클에서도 도달할 수 있는 정점에는 유한한 최단 거리가 없으므로, 이 불변식을 적용할 수 없다.

음수 간선에서 Dijkstra가 깨지는 예



1단계: S 확정 → A = 3, B = 4

2단계: A(3) 확정 → 이미 final이라 믿음

3단계: B(4) 확정 → B→A로 A = 2

실제 최단은 S→B→A = 2인데 A는 3으로 확정됨
A 뒤의 노드까지 연쇄로 틀어짐

negative cycle이면?

한 바퀴 돌 때마다 길이가 줄어 최단 경로가 정의되지 않음
→ 없다고 가정, 음수 간선만 허용 (Bellman-Ford)

---

2. 절차

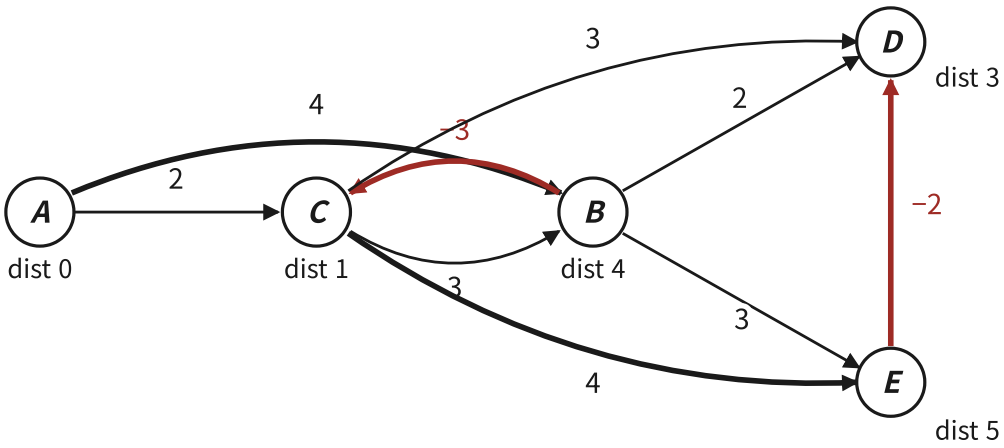
모든 간선을 $|V| - 1$번 순회하며 갱신한다.

```
for i = 1 to |V| - 1
  for all e in E
    update(e)
```

라운드별 손계산: 슬라이드 28 그래프

표의 각 라운드는 직전 라운드의 스냅샷을 이용해 계산하는 동기식 라운드표이다. 반면 실제 슬라이드 코드는 in-place 갱신을 사용한다. In-place 방식에서는 같은 라운드 안에서 이미 갱신된 값을 바로 이용해 다음 간선을 계산할 수 있다.

Bellman-Ford (시작 A) — 빨강 = 음수 간선 · 굵은 선 = shortest path tree



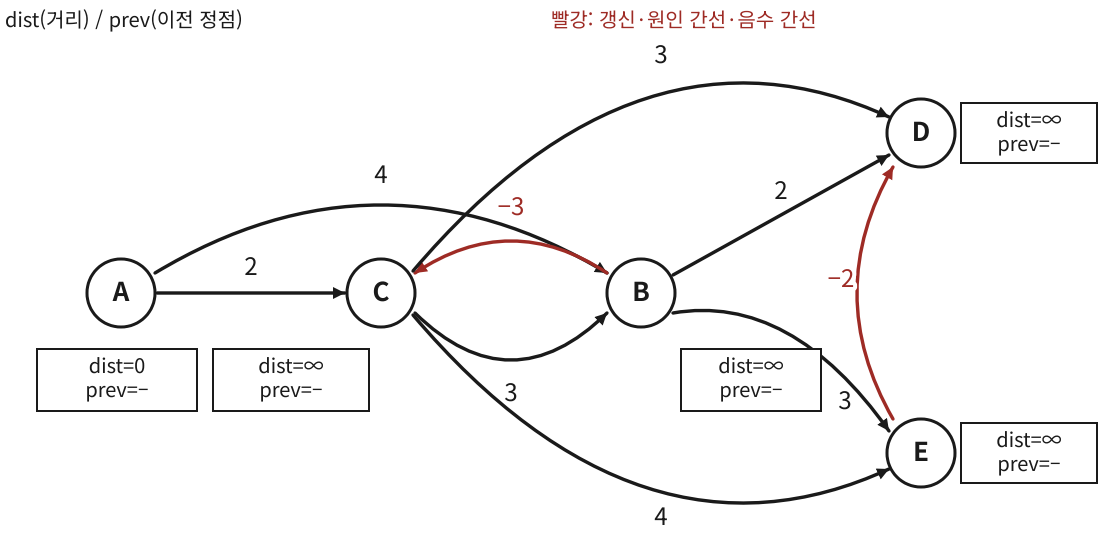
라운드 i 끝 = 간선 i개 이하 최단 · B 1, C 2, E 3, D 4라운드에 final
Dijkstra 그대로 돌리면 D, E가 틀림

라운드	A	B	C	D	E
0	0 / nil	∞ / nil	∞ / nil	∞ / nil	∞ / nil
1	0 / nil	4 / A	2 / A	∞ / nil	∞ / nil
2	0 / nil	4 / A	1 / B	5 / C	6 / C
3	0 / nil	4 / A	1 / B	4 / C	5 / C
4	0 / nil	4 / A	1 / B	3 / E	5 / C

각 셀은 $dist/prev$이다.

- 노드별 final 라운드: B는 1라운드(A→B), C는 2라운드(A→B→C), E는 3라운드, D는 4라운드에 final이 된다.
- 간선이 i개 이하인 최단 경로는 동기식 라운드 i가 끝날 때 정확하게 계산된다.
- 이 그래프의 shortest path tree는 A→B (4), B→C (−3), C→E (4), E→D (−2)이다.
- B와 C 사이의 왕복 사이클 가중치는 $3 + (-3) = 0$이므로 음수 사이클이 아니다.
- 같은 그래프에 Dijkstra를 그대로 적용하면 C가 거리 2로 확정된 뒤 거리 1로 낮아지는 상황을 처리하지 못한다. 그 결과 Dijkstra는 D 5, E 6을 반환하지만, 정답은 D 3, E 5이다.

Bellman-Ford (시작 A) — 음수 간선 최단 경로



각 라운드의 갱신은 **직전 라운드 스냅샷**으로 계산(동기식). 셀은 dist / prev.

라운드	A	B	C	D	E
R0	0 / –	∞ / –	∞ / –	∞ / –	∞ / –

초기화. 시작점 A의 dist는 0, 나머지는 ∞이다. 표의 각 셀은 dist(거리) / prev(이전 정점)이다. 빨강은 이번 라운드에 갱신된 노드·원인 간선과 음수 간선을 나타낸다. **동기식** 라운드 i 끝의 dist는 간선 i개 이하를 사용하는 경로 중 최솟값이다.

◀ 이전 1 / 7 다음 ▶

라운드별 해설

- 라운드 2: B→C로 C가 2에서 1로 내려간다. 음수 간선 덕분이다. D와 E는 C의 이전 값 2를 기준으로 계산되므로 각각 5와 6이다.
- 라운드 3: D는 C 경유 4와 E 경유 $6 - 2 = 4$가 동률이다. 엄격한 비교만 사용하므로 $prev(D) = C$가 유지된다.
- 라운드 4: E의 값 5가 반영되어 E→D를 통해 D가 3으로 내려간다.
- 동기식에서는 라운드 i 종료 시 간선 i개 이하 경로의 최단 거리가 정확하다. In-place에서는 라운드별 dist/prev 값이 순서에 따라 달라질 수 있으나, 음수 사이클이 없으면 |V|−1번 순회 후 모든 도달 가능 정점의 최단 거리가 정확해진다는 보장은 유지된다. |V|번째 순회에서도 갱신이 발생하면 도달 가능한 음수 사이클이 존재한다. 이 그래프는 간선을 알파벳 순으로 처리하면 1라운드에 수렴한다.
- 음수 사이클 검사: $|V|$번째 라운드에도 $dist$가 줄어들면 출발점에서 도달 가능한 음수 사이클이 존재한다. 이 그래프에서는 5번째 라운드에 갱신이 없다.

3. 정확성

최단 경로의 성질

1. 최단 경로의 모든 prefix도 최단 경로이다. 그렇지 않다면 해당 prefix를 더 짧은 경로로 대체하여 전체 경로를 더 짧게 만들 수 있다.
2. 음수 사이클이 없으면 최단 경로 중 단순 경로를 하나 선택할 수 있다. 경로에 포함된 사이클은 가중치가 음수가 아니므로 제거해도 길이가 증가하지 않는다. 따라서 정점이 $|V|$개일 때 간선이 $|V|-1$개 이하인 최단 경로가 존재한다.
3. 최단 경로가 정확히 i개의 간선으로 이루어졌다면, 마지막 간선 바로 앞의 정점까지의 최단 거리가 정확해진 뒤 마지막 간선을 갱신하면 목적 정점의 거리도 정확해진다.

라운드별 귀납: 초기에는 출발점의 거리만 0이고 나머지는 ∞이다. 동기식 갱신에서 라운드 i가 끝나면 간선이 i개 이하인 최단 경로의 거리가 정확하다. 단순 최단 경로에는 최대 $|V|-1$개의 간선만 필요하므로, $|V|-1$번의 갱신 후에는 음수 사이클이 없는 경우 모든 도달 가능한 정점의 최단 거리가 정확해진다. In-place 갱신에서는 같은 라운드 안에 이미 갱신된 값을 재사용할 수 있어 더 빨리 수렴할 수 있다. 그러나 간선을 $|V|-1$번 순회하면 필요한 최단 거리들을 계산할 수 있다는 보장은 유지된다.

---

4. 복잡도

Bellman-Ford는 모든 간선을 $|V|-1$번 검사하므로 시간 복잡도는 $O(|V||E|)$이다. Dijkstra보다 느리지만 음수 간선을 허용한다.

항목	Dijkstra	Bellman-Ford
간선 길이	음수가 아닌 가중치(양수만 가정해도 정확)	음수 가능, 음수 사이클 제외
방식	가장 가까운 노드를 하나씩 확정	모든 간선을 $|V|-1$번 갱신
자료구조	Priority queue	별도의 priority queue 불필요
확정 시점	deletemin 때 해당 노드의 dist	라운드 끝에 간선 개수 제한에 따른 최단 거리 확정
시간 복잡도	$O((\|V\|+\|E\|)\log\|V\|)$, binary heap 기준	$O(\|V\|\|E\|)$

보충 · Shortest Reliable Paths (간선 수 제한 최단 경로)

문제 정의

입력은 그래프 $G = (V, E)$, 출발 정점 s, 도착 정점 t, 정수 k이다. 목표는 **최대 k개의 간선을 사용하는 s에서 t까지의 최단 경로**를 찾는 것이다.

간선 수 제한이 있는 문제에서는 경로의 길이뿐 아니라 사용한 간선 수도 고려해야 한다. Dijkstra 알고리즘은 경로에 포함된 간선 수를 기억하지 않으므로, 간선 수 제한을 직접 반영하는 데 적합하지 않다. 따라서 부분 문제를 정의할 때 허용된 간선 수와 최단 경로 길이에 관한 정보를 함께 유지해야 한다.

부분 문제와 점화식

$dist(v, i)$를 s에서 v까지 정확히 i개의 간선을 사용하여 도달하는 최단 경로의 길이라고 하자.

초기 조건은 다음과 같다.

- $dist(s, 0) = 0$
- $dist(v, 0) = \infty$ $(v \neq s)$

$i \geq 1$일 때, 점화식은 s를 포함한 모든 정점 v에 적용되며 다음과 같다.

$dist(v, i) = \min_{(u, v) \in E} \{ dist(u, i - 1) + \ell(u, v) \}$

여기서 $\ell(u, v)$는 간선 (u, v)의 길이이다. 이 점화식은 v로 들어오는 각 간선 (u, v)를 고려한다. 마지막 간선이 (u, v)라면, 그 직전까지는 정확히 $i - 1$개의 간선을 사용하여 u에 도달해야 한다. 따라서 이전 단계의 $dist(u, i - 1)$에 마지막 간선의 길이 $\ell(u, v)$를 더하고, 가능한 후보 중 최솟값을 선택한다.

최종 답은 $\min_{i \leq k} dist(t, i)$이다.

동적 계획법과 Bellman-Ford의 관계

이 문제는 허용 간선 수를 단계별로 늘려 가며 최단 경로 길이를 계산하는 동적 계획법 문제이다. 각 단계의 값은 바로 이전 단계의 값과 간선의 길이를 이용하여 계산한다.

이는 Bellman-Ford의 반복적인 간선 완화와 같은 관점으로 이해할 수 있다. 간선을 따라 가능한 경로 길이를 갱신하되, 여기서는 허용된 간선 수 i를 상태에 명시적으로 포함한다.

또한 각 상태 $dist(v, i)$를 정점으로 생각하고, 원래 그래프의 간선 (u, v)에 대해 이전 단계의 상태 $(u, i - 1)$에서 현재 단계의 상태 (v, i)로 연결하면, 단계 i가 증가하는 방향으로만 이동하는 DAG를 얻는다. 따라서 이 상태 그래프에서는 단계 순서에 따라 점화식을 계산할 수 있다.

슬라이드 예제의 손계산 추적

슬라이드에 제시된 경로와 길이는 다음과 같다.

허용 간선 수 k	최단 경로	경로 길이
2	SDT	8
3	SCDT	6
4	SABDT	5

허용 간선 수가 2, 3, 4로 증가할 때, 예제의 최단 경로 길이는 8, 6, 5로 감소한다. 간선 수 제한이 커지면 이전에 허용되던 경로도 계속 허용되므로 최적 경로 길이가 나빠질 필요가 없다. 이 예제는 허용 간선 수가 증가하면서 더 짧은 경로를 선택할 수 있음을 보여 준다.

슬라이드에는 예제 그래프의 모든 간선 가중치가 제시되어 있지 않으므로, 위 경로와 길이를 넘어서는 중간 거리 값이나 간선별 계산은 추가하지 않는다.

시간 복잡도

각 단계 i = 1, ⋯, k에서 모든 간선 (u, v) ∈ E를 확인하여 점화식의 최솟값을 계산한다. 각 단계의 계산에는 O(|E|) 시간이 필요하고, 이러한 단계를 k번 수행하므로 전체 시간 복잡도는 O(k|E|)이다. 또한 dist(v, i)는 모든 정점 v와 모든 단계 i = 0, ⋯, k에 대한 값을 유지하므로 공간 복잡도는 O(k|V|)이다.

DAG — 위상 순서와 최단 경로

- **topological ordering** (위상 순서) — 모든 간선 (u, v)에서 u가 v보다 앞에 오는 노드 순서. 줄로 펴면 모든 간선이 왼쪽 → 오른쪽
- **DAG ⇔ topological ordering이 존재**. 순서는 유일하지 않음
- 만드는 법 ① source(in-degree 0)를 간선과 함께 제거하는 것을 반복 — 남은 그래프도 DAG ② DFS post 내림차순 (앞 장)

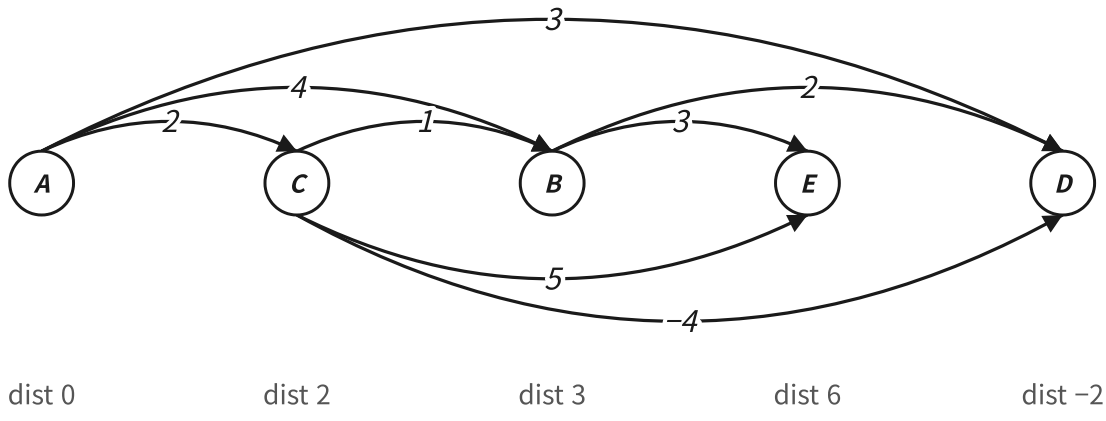
- 슬라이드 그래프 (A→B 4, A→C 2, A→D 3, C→B 1, C→D −4, C→E 5, B→D 2, B→E 3)의 in-degree 추적: 0단계 A 0, B 2, C 1, D 3, E 2 → A 제거 → C 제거 → B 제거 → E (또는 D) 제거 → D

제거. **제거 순서 A, C, B, E, D**

- 순서는 유일하지 않음 — D, E를 맞바꾼 A, C, B, D, E도 가능 (이 그래프는 정확히 2개). DFS post 내림차순(이웃은 알파벳 순)으로도 A, C, B, E, D가 나옴 (post: A 10, C 9, B 7, E 6, D 4)
- cycle이 있으면 제거하다 막힘 — 슬라이드 17 그래프는 모든 노드 in-degree ≥ 1이라 첫 단계부터 제거 불가
- in-degree 카운터를 갱신하며 제거하면 $O(|V| + |E|)$. DAG 여부 자체는 DFS back edge로 확인

DAG에서의 최단 경로

DAG 최단 경로 — 선형화 순서 A, C, B, E, D (음수 간선 OK) · 아래 = 최종 dist



노드를 처리할 때 들어오는 간선은 이미 전부 반영됨 → dist가 final · O(|V|+|E|)

들어오는 간선만 보면 점화식: dist(D) = min{dist(B) + 2, dist(C) − 4, dist(A) + 3} → = dynamic programming

부등호를 뒤집으면 longest path도 같은 선형 시간

- 선형화 순서대로 노드를 보며 나가는 간선을 update. 노드를 처리할 때 들어오는 간선은 모두 앞 노드에서 와서 이미 반영됨 → **dist가 final**. priority queue 불필요, **음수 간선 OK**
- 시간 $O(|V| + |E|)$.
- 손계산 (출발 A): A 처리 (C 2, B 4, D 3) → C 처리 (B 3, E 7, D −2) → B 처리 (E 6, D는 5 > −2 그대로) → E, D 처리 (나가는 간선 없음). 최종 A 0, C 2, B 3, E 6, D −2 — 같은 그래프에서 Bellman-Ford(라운드 $|V| - 1$회)를 돌려도 이 최종 dist와 같음. DAG라 위상 순서 한 번 훑기($O(|V| + |E|)$)로 충분하고, 음수 간선(C→D −4)이 있어도 cycle이 없어 정확히 계산됨
- 들어오는 간선만 보면 **점화식 = dynamic programming**:

$$\text{dist}(v) = \min_{(u,v)\in E} \{\text{dist}(u) + \ell(u, v)\}$$

- 부등호를 뒤집으면 (dist[u] + ℓ > dist[v], 초기값 $-\infty$) **longest path** — 일반 그래프에선 어렵지만 DAG에서는 같은 선형 시간. 슬라이드 31 그래프의 longest S → E = 14
- 슬라이드 31 그래프 (간선 S→A 1, S→C 2, C→A 4, A→B 6, B→D 1, C→D 3, B→E 2, D→E 1, 처리 순서 S, C, A, B, D, E) 최단 추적: S 처리 (A 1, C 2) → C 처리 (D 5; A는 2 + 4 = 6 > 1 그대로) → A 처리 (B 7) → B 처리 (E 9) → D 처리 (E 6). dist(D) = min{7 + 1, 2 + 3} = 5. 최단 S → E: S → C → D → E = 2 + 3 + 1 = 6

동적 계획법으로 풀리는 최단 경로 문제

모든 쌍 최단 경로 (all pairs)

1. 조건

모든 정점 쌍 (i, j)에 대해 최단 경로를 구한다. 한 출발점에서의 최단 경로 알고리즘을 각 정점에서 반복하거나, 동적 계획법인 Floyd-Warshall을 사용할 수 있다.

- Unweighted 그래프: BFS.
- Acyclic 그래프(DAG): 위상 순서로 훑기.
- 음수가 아닌 가중치 그래프: Dijkstra.
- 일반 가중치 그래프: Bellman-Ford. 음수 간선을 허용하되, 최단 거리가 정의되도록 음수 사이클이 없어야 한다.

2. 절차

방법 1: 단일 출발점 알고리즘 반복 — 각 정점을 출발점으로 삼아 해당 알고리즘을 한 번씩 실행한다.

그래프	알고리즘	한 번 실행	모든 정점에서 실행
Unweighted	BFS	$O(\|V\| + \|E\|)$	$O(\|V\|(\|V\| + \|E\|))$
Acyclic	위상 순서로 훑기	$O(\|V\| + \|E\|)$	$O(\|V\|(\|V\| + \|E\|))$
음수가 아닌 가중치	Dijkstra (binary heap)	$O((\|V\| + \|E\|) \log \|V\|)$	$O(\|V\|(\|V\| + \|E\|) \log \|V\|)$
일반 가중치	Bellman-Ford	$O(\|V\|\|E\| + \|V\|)$	$O(\|V\|^2\|E\| + \|V\|^2)$

연결 그래프에서는 $|E| = \Omega(|V|)$이므로 기존의 간략한 표현을 사용할 수 있다. 단순 그래프의 최악 경우 $|E| = O(|V|^2)$를 대입하면 BFS와 DAG는 $O(|V|^3)$, Dijkstra는 $O(|V|^3 \log |V|)$, Bellman-Ford는

$O(|V|^4)$ 등의 상한을 얻는다.

방법 2: Floyd-Warshall (DP) — 중간 정점으로 허용할 수 있는 집합을 단계별로 늘린다. 정점의 순서를 A, B, C, D, E로 두고, $dist(i, j, k)$를 처음 k개의 정점만 중간 정점으로 허용했을 때 i에서 j까지 가는 최단 경로 길이라고 정의한다. k번째 정점을 중간 정점으로 허용할 때 두 경우를 비교한다.

1. k를 지나지 않는 경로: 이전 값 그대로.
2. k를 지나는 경로: $i \to k$와 $k \to j$를 연결한다.

$$dist(i, j, k) = \min\{dist(i, j, k-1),\ dist(i, k, k-1) + dist(k, j, k-1)\}$$

초기값은 다음과 같다.

$$dist(i, j, 0) = \begin{cases} 0 & i = j \\ \ell(i, j) & \text{간선 } (i, j)\text{가 있으면} \\ \infty & \text{그 외} \end{cases}$$

k를 가장 바깥쪽 루프로 두어야 한다. 각 단계는 이전 단계의 값만 사용하기 때문이다.

초기 거리 행렬 ($k = 0$)

	A	B	C	D	E
A	0	4	2	∞	∞
B	∞	0	−3	2	3
C	∞	3	0	3	4
D	∞	∞	∞	0	∞
E	∞	∞	∞	−2	0

단계별로 바뀌는 칸

- $k = A$: 바뀌는 칸 없음. A로 들어오는 간선이 없다.
- $k = B$: $d(A, C)$: $2 \to 1$, $d(A, D)$: $\infty \to 6$, $d(A, E)$: $\infty \to 7$.
- $k = C$: $d(A, D)$: $6 \to 4$, $d(A, E)$: $7 \to 5$, $d(B, D)$: $2 \to 0$, $d(B, E)$: $3 \to 1$.
- $k = D$: 바뀌는 칸 없음.
- $k = E$: $d(A, D)$: $4 \to 3$, $d(B, D)$: $0 \to -1$, $d(C, D)$: $3 \to 2$.

최종 거리 행렬 ($k = 5$)

	A	B	C	D	E
A	0	4	1	3	5
B	∞	0	−3	−1	1
C	∞	3	0	2	4
D	∞	∞	∞	0	∞
E	∞	∞	∞	−2	0

3. 정확성

단일 출발점 알고리즘 반복: 각 출발점에서 선택한 알고리즘이 정확하다면, 모든 출발점에서 실행한 결과는 모든 정점 쌍의 최단 거리를 제공한다. 단, 해당 알고리즘의 적용 조건을 만족해야 한다.

Floyd-Warshall의 귀납적 정확성

- 초기 단계는 직접 간선만 허용하므로 초기 거리 행렬이 정확하다.
- $k-1$단계까지 정확하다고 가정하자. k단계의 최단 경로는 k를 중간 정점으로 사용하지 않거나, k를 지나 두 구간으로 나뉘는 경우 중 하나이다.
- 두 경우의 길이 중 최솟값을 선택하므로 k단계도 정확하다.
- 마지막 단계에서는 모든 정점을 중간 정점으로 허용하므로 전체 최단 거리를 얻는다.
- 모든 쌍의 최단 거리가 정의되도록 음수 사이클이 없다고 가정한다.
- 예를 들어 $k = C$에서 $d(A, D) = 4$는 $A \to B \to C \to D$의 길이 $4 - 3 + 3$이다. $k = E$에서는 $d(A, D) = 3$이 되며, 경로는 $A \to B \to C \to E \to D$, 길이는 $4 - 3 + 4 - 2$이다.
- 최종 A행은 Bellman-Ford 결과인 B 4, C 1, D 3, E 5와 일치한다.

4. 복잡도

- 반복 실행 방법은 각 단일 출발점 알고리즘의 시간 복잡도에 $|V|$를 곱한 비용이 든다.
- Floyd-Warshall은 정점 쌍마다 중간 정점 단계를 모두 검사하므로 시간 복잡도는 $O(|V|^3)$이다.
- 모든 단계의 행렬을 보관하면 공간 복잡도는 $O(|V|^3)$이다. 이전 단계의 행렬만 사용하므로 하나의 표를 덮어쓰면 공간을 $O(|V|^2)$로 줄일 수 있다.

TSP

1. 조건

- 입력: 도시 집합과 도시 사이의 거리. 출발 도시는 1로 고정한다.
- 목표: 모든 도시를 정확히 한 번씩 방문하고 출발 도시 1로 돌아오는 최단 tour를 구한다.
- 일반적인 TSP 최적화 문제는 NP-hard이다. 다항 시간에 정확히 푸는 알고리즘은 알려져 있지 않으며, 그러한 알고리즘이 존재한다면 $P = NP$가 성립한다. 대응하는 결정 문제는 NP-complete이다.

2. 절차

부분문제 정의: $C(S, j)$를 도시 집합 S에 속한 도시들을 각각 정확히 한 번씩 방문하면서, 1에서 출발하여 j에서 끝나는 최단 경로의 길이라고 정의한다. S에는 출발 도시 1과 마지막 도시 j가 포함된다. 기본값은 $C(\{1, j\}, j) = d(1, j)$이다. 그 외의 부분문제는 방문한 도시 수가 작은 순서부터 채운다.
경계 조건으로 $C(T, 1) = \infty$ ($T \neq \{1\}$)을 둔다.

$$C(S, j) = \min_{i \in S \setminus \{1, j\}} \{C(S \setminus \{j\}, i) + d(i, j)\}$$

마지막에는 각 가능한 마지막 도시에서 1로 돌아오는 비용을 더하고 최솟값을 선택한다.

$$\min_{j \neq 1} \{C(V, j) + d(j, 1)\}$$

손계산 예시

S	j	계산	값
{1, 2}	2	$d(1, 2)$	10
{1, 3}	3	$d(1, 3)$	15
{1, 4}	4	$d(1, 4)$	20
{1, 2, 3}	2	$C(\{1, 3\}, 3) + d(3, 2) = 15 + 35$	50
{1, 2, 3}	3	$C(\{1, 2\}, 2) + d(2, 3) = 10 + 35$	45
{1, 2, 4}	2	$C(\{1, 4\}, 4) + d(4, 2) = 20 + 25$	45
{1, 2, 4}	4	$C(\{1, 2\}, 2) + d(2, 4) = 10 + 25$	35

{1, 3, 4}	3	$C(\{1,4\},4)+d(4,3)=20+30$	50
{1, 3, 4}	4	$C(\{1,3\},3)+d(3,4)=15+30$	45
{1, 2, 3, 4}	2	$\min\{85,70\}$	70
{1, 2, 3, 4}	3	$\min\{80,65\}$	65
{1, 2, 3, 4}	4	$\min\{75,75\}$	75

출발 도시로 돌아오기

- $j=2$: $70+10=80$
- $j=3$: $65+15=80$
- $j=4$: $75+20=95$

따라서 최단 tour 길이는 80이다. $1\to2\to4\to3\to1=10+25+30+15=80$. 모든 tour를 직접 비교해도 최단 길이는 80이다.

3. 정확성

부분문제 $C(S,j)$에서 최적 경로의 마지막 간선 직전 정점을 i라고 하자. 그러면 경로는 도시 집합 $S\setminus\{j\}$를 방문하고 i에서 끝나는 경로에 간선 (i,j)를 추가한 형태이다. 이전 부분 경로가 최적이 아니라면 그 부분을 더 짧은 경로로 대체하여 전체 경로를 더 짧게 만들 수 있다. 따라서 최적 경로의 각 부분 경로도 최적이어야 한다. 가능한 직전 정점 i를 모두 비교하여 최솟값을 선택하면 $C(S,j)$가 정확하게 계산된다. 모든 도시를 방문한 뒤 마지막 도시 j에서 출발 도시 1로 돌아오는 간선까지 고려하고, 가능한 모든 j에 대해 최솟값을 선택하면 전체 최단 tour를 얻는다.

4. 복잡도

출발 도시 1을 포함하는 부분문제에서 $j\neq1$로 세면 부분문제 수는 정확히 $(n-1)2^{n-2}$이다. 각 부분문제에서는 가능한 직전 도시 후보를 검사한다. 따라서 전체 시간 복잡도는 $O(n^2 2^n)$, 공간 복잡도는 $O(n2^n)$이다. 모든 tour를 직접 열거하는 방법보다 훨씬 작지만, 여전히 지수 시간이다.

부분문제·전이·tour 수 비교

도시 수 n	부분문제 수	전이 (후보 비교) 수	Tour 수 $(n-1)!$
4	12	12	6
5	32	48	24

10	2,304	9,216	362,880
15	114,688	745,472	약 8.7×10^{10}
20	4,980,736	44,826,624	약 1.2×10^{17}

표의 전이 후보 비교 수는 기저 상태 C({1, j}, j) = d(1, j)를 제외하고, 각 비기저 상태에서 점화식에 따라 직전 도시 후보 i ∈ S \ {1, j}를 검사하는 횟수를 합산한 값이다. 따라서 n ≥ 3일 때 전이 후보 비교 수는 $(n-1)(n-2)2^{n-3}$이다. 출발 도시를 고정한 모든 tour의 수는 (n−1)!이다. TSP 동적 계획법의 부분문제 수는 $(n-1)2^{n-2}$이며, 시간복잡도는 $O(n^2 2^n)$, 공간복잡도는 $O(n2^n)$이다. 이 점화식과 복잡도는 그대로 유지된다.

도시 수가 작을 때는 구현 방식과 상수 비용에 따라 모든 tour를 직접 열거하는 방법도 경쟁력 있을 수 있다. 출발 도시를 고정하면 tour는 (n−1)!개이며, 각 tour의 비용을 직접 계산하는 단순한 방법의 시간복잡도는 $O(n(n-1)!)$이다. 반면 동적 계획법의 시간복잡도는 $O(n^2 2^n)$이다. Tour의 개수와 DP의 부분문제 수 또는 전이 후보 비교 횟수는 서로 다른 연산 단위이므로, 그 숫자만으로 어느 방법이 실제로 더 빠른지 단정해서는 안 된다. 도시 수가 증가하면 factorial growth를 갖는 완전 열거보다 $O(n^2 2^n)$ 동적 계획법이 점근적으로 유리하다.

출발 도시 1을 고정한 방문 순서열의 수는 $(n-1)!$이다. 대칭 TSP에서 역방향을 같은 tour로 간주하면 $n > 2$일 때 서로 다른 무방향 tour의 수는 $(n-1)!/2$이다.

연습문제

문제 1

슬라이드 22의 방향 그래프에서 출발점을 B로 바꿔 Dijkstra를 돌리고 확정 순서와 dist를 구하시오. A는 어떻게 되는가.

힌트 보기

deletemin으로 가장 작은 노드를 꺼내 확정하고 나가는 간선만 update. 간선은 화살표 방향으로만 따라감 — 들어오는 간선이 없는 노드는 영원히 ∞.

정답 보기

개념은 위 힌트 참고.

① B의 나가는 간선: B→C 3, B→D 2, B→E 3. deletemin B(0) → C = 3, D = 2, E = 3

② deletemin D(2): 나가는 간선 없음 → 그대로

③ deletemin C(3): C→B, C→D, C→E 모두 더 길어 갱신 없음

④ deletemin E(3): E→D = 3 + 1 = 4 > 2 → 갱신 없음

⑤ A로 들어오는 간선이 없음 → dist(A) = ∞ (도달 불가)

최종 답 **확정 순서 B, D, C, E (C, E 동률은 무관) · dist: B 0, D 2, C 3, E 3, A ∞**

문제 2

슬라이드 28 그래프에서 C→B의 길이를 3에서 −3으로 바꾸면 어떻게 되는가. Bellman-Ford 라운드별 B, C 값으로 확인하시오.

힌트 보기

① B→C −3과 C→B −3이 왕복 cycle을 만듦

② 라운드 i는 라운드 $i-1$의 값만 읽어 update

③ $|V|$번째 라운드에도 값이 줄어들면 negative cycle

정답 보기

개념은 위 힌트 참고.

① B→C→B 길이 = −3 + (−3) = −6 → negative cycle

② 라운드별 B: ∞, 4, −1, −2, −7, −8 (라운드 0 → 5)

③ 라운드별 C: ∞, 2, 1, −4, −5, −10

④ 5번째(= $|V|$) 라운드에도 값이 줄어듦 → negative cycle 검출

최종 답 **negative cycle 발생 → 계속 돌면 −∞라 최단 경로가 정의되지 않음**

문제 3

슬라이드 29의 DAG에서 출발점을 C로 바꿔 선형화 순서(A, C, B, E, D) 위에서 최단 경로를 구하시오.

힌트 보기

- 순서상 앞 노드에서 뒤 노드로만 감
- 출발점보다 순서가 앞인 노드는 도달 불가
- dist = ∞인 노드는 처리해도 갱신 없음

정답 보기

개념은 위 힌트 참고.

① A 처리: dist(A) = ∞라 건너뜀 (A는 C보다 앞이라 도달 불가)

② C 처리: B = 1, E = 5, D = −4

③ B 처리: E = 1 + 3 = 4 (D는 1 + 2 = 3 > −4 → 그대로)

④ E, D 처리: 나가는 간선 없음 → 그대로

⑤ E의 prev는 B (C→B→E = 4 < 직접 5)

최종 답 **C 0, B 1, E 4, D −4, A ∞**

문제 4

슬라이드 31의 DAG에서 부등호를 뒤집어 S에서 E까지 longest path를 구하시오.

힌트 보기

초기값 −∞, dist(S) = 0, 갱신 조건을 dist[u] + ℓ > dist[v]로. 선형화 순서(S, C, A, B, D, E)는 그대로.

정답 보기

개념은 위 힌트 참고.

① S 처리: A = 1, C = 2

② C 처리: A = max(1, 2 + 4) = 6, D = 5

③ A 처리: B = 6 + 6 = 12

④ B 처리: D = max(5, 12 + 1) = 13, E = 12 + 2 = 14

⑤ D 처리: E = max(14, 13 + 1) = 14 → 그대로

최종 답

longest S → E = 14 · S-C-A-B-E (2+4+6+2)와 S-C-A-B-D-E (2+4+6+1+1)가 동률

문제 5

① 슬라이드 28 그래프의 Floyd-Warshall에서 $k =$ A일 때 값이 바뀌는 칸이 없는 이유와 $k =$ B일 때 바뀌는 칸을 구하시오

② TSP에서 $n = 5$일 때 C(S, j) 부분문제 수($j \neq 1$)와 모든 tour 수를 비교하시오

힌트 보기

- k를 지나는 후보 = d(i, k) + d(k, j). d(i, k)가 ∞면 후보 자체가 무의미
- TSP 부분문제 수(j ≠ 1) = $(n-1)2^{n-2}$

- tour 수 = $(n-1)!$

정답 보기

개념은 위 힌트 참고.

① k = A: A로 들어오는 간선이 없어 d(i, A, 0) = ∞ (i ≠ A) → d(i, A) + d(A, j) 후보가 전부 ∞라 바뀌는 칸 없음

② k = B: d(A,C) 2 → 1 (A→B→C = 4 − 3), d(A,D) ∞ → 6 (4 + 2), d(A,E) ∞ → 7 (4 + 3)

③ TSP n = 5: 부분문제 수는 $(n-1)2^{n-2} = 4 \times 8 = 32$개이고, 전이 후보 비교 수는 $(n-1)(n-2)2^{n-3} = 4 \times 3 \times 4 = 48$회이다.

④ n = 5일 때 DP 부분문제는 32개이고, 점화식의 전이 후보 비교는 48회이며, 출발 도시를 고정한 tour는 24개이다. 이 숫자들은 각각 부분문제 수, 후보 비교 횟수, tour 수를 뜻한다. 실제 실행 시간의 우열은 이 숫자만으로 확정할 수 없다.

최종 답

k = A는 변화 없음 · k = B는 (A,C) (A,D) (A,E) 3칸 · n = 5일 때 DP 부분문제는 32개이고, 점화식의 전이 후보 비교는 48회이며, 출발 도시를 고정한 tour는 24개이다